VISIONINSPECT AI

Final Project Documentation – Milestone 4

Testing • Validation • Deployment • Documentation • Final Demonstration

Project Information	Details
Project	VisionInspect AI
Milestone	Milestone 4 – Testing, Deployment & Documentation
Backend	Python, FastAPI
Frontend	React.js / Vite / Nginx
Database	PostgreSQL
AI/CV	PyTorch, Torchvision, ResNet18 feature extraction, OpenCV, NumPy, Scikit-learn
Dataset	MVTec Anomaly Detection – 15 categories
Deployment	Docker Compose on Microsoft Azure VM
Version Control	Git + GitHub
Status	Live end-to-end deployment completed

1. Executive Summary

VisionInspect AI is an AI-powered manufacturing quality inspection platform that accepts product images, performs computer-vision-based anomaly detection and defect classification, assigns severity, generates a quality decision, stores inspection records, and presents manufacturing analytics and reports.

Milestone 4 focuses on testing and validation, inspection-performance optimization, dashboard/report improvements, Docker and cloud deployment, technical documentation, and final end-to-end demonstration. The completed platform is deployed on an Azure Linux virtual machine.

2. Milestone 4 Requirements and Completion Status

The project specification defines Milestone 4 as Week 7–8: Testing, Deployment & Documentation. It requires validation of detection/classification quality, performance optimization, dashboard/report improvements, Docker/cloud deployment, technical documentation, and a complete demonstration.

Requirement	Implementation / Evidence	Status
Validate detection and classification	Hold-out evaluation across 15 MVTec categories with separate validation/test data; anomaly and classification metrics recorded.	Completed
Optimize CV models/performance	Threshold tuning evaluated for difficult categories; CPU-only PyTorch deployment configured for Azure execution.	Completed
Improve dashboard/report quality	Dashboard, inspection history, analytics, severity views and PDF/CSV reports verified in deployment.	Completed
Docker/cloud deployment	React/Nginx frontend, FastAPI backend and PostgreSQL deployed using Docker Compose on Azure VM.	Completed
Documentation/presentation	Final technical documentation prepared; presentation/demo material can accompany submission.	Completed
Complete demonstration	Login → upload → inspect → result → history → analytics → reports verified end-to-end.	Completed

3. Project Objectives

- Automate visual quality inspection of manufactured products using computer vision.
- Detect anomalous or defective product images from the MVTec dataset.
- Classify detected defects and provide a quality-control decision.
- Assign severity levels based on defect characteristics and confidence.
- Provide inspection history, analytics, trends and production-quality reports.
- Support role-based access for Quality Engineers and Factory Supervisors.
- Deploy the complete system as a containerized cloud application.

4. System Architecture

The deployed platform uses a three-service Docker architecture. The React/Vite frontend is served by Nginx. Nginx also reverse-proxies /api requests to FastAPI. The backend performs authentication,

inspection processing, AI inference, severity assessment, analytics and reporting. PostgreSQL stores users and inspection records, while the MVTec dataset is mounted into the backend container.

Layer	Technology	Responsibility
Presentation	React.js / Vite / Nginx	Login, dashboards, upload, inspection results, analytics and reports
Application/API	FastAPI / Uvicorn	REST APIs, authentication, inspection workflow, analytics and reports
AI/CV	PyTorch / Torchvision / ResNet18 / OpenCV	Feature extraction, anomaly scoring and defect classification
Database	PostgreSQL	Users, roles and inspection history
Dataset	MVTec AD	15 categories and reference images
Deployment	Docker Compose / Azure VM	Container orchestration and cloud hosting

4.1 End-to-End Workflow

1. Authenticate using the VisionInspect AI login system.
2. Open New Inspection and upload a supported product image.
3. Select the product category and inspection mode.
4. Frontend sends the image to the FastAPI backend through Nginx.
5. Backend validates the image and performs computer-vision processing.
6. ResNet18 features are extracted and compared with category reference images.
7. Anomaly detection determines normal/defective status.
8. Classification estimates the most likely defect type using category-specific reference/prototype features.
9. Severity and quality-control logic generate the final result.
10. Result is stored in PostgreSQL and returned to the frontend.
11. Inspection History, Analytics and Reports use the stored inspection data.

5. Functional Modules

Module	Implemented Features
User Management	Registration, login, JWT authentication and role-based access
Image Acquisition	Drag/drop upload, JPG/PNG/BMP support, validation and 10 MB limit
Image Processing	Image validation, preprocessing and feature extraction
Defect Detection	Anomaly detection using ResNet18 features and reference-image comparison
Defect Classification	Category-specific defect classification/prototype comparison
Severity & Quality Control	Severity level, pass/fail/review decision and recommendations
Inspection History	Stored records, search/history view and detailed results
Manufacturing Analytics	Pass rate, defect rate, critical rate, trends and defect distribution
Reporting	Production-quality PDF and inspection CSV export
Role-Based Dashboards	Quality Engineer and Factory Supervisor views

6. AI and Computer Vision Methodology

The deployed inspection engine uses a pretrained ResNet18 network as a feature extractor. It does not claim end-to-end fine-tuning of ResNet18. Product images are represented in feature space and compared with reference images from the selected MVTec category. The resulting anomaly score is evaluated against a category-specific threshold.

6.1 Anomaly Detection

- Reference images are taken from the selected category's train/good directory.
- ResNet18 converts images into deep feature representations.
- The inspected image is compared against reference features using feature-space distance.
- A threshold derived from normal-reference scores determines anomaly/defect status.
- Category-specific threshold multipliers were evaluated for difficult categories.

6.2 Defect Classification

- Category-specific reference/prototype features are maintained for defect classes.
- The query image is compared against class prototypes to estimate the defect type.
- Classification was evaluated separately from anomaly detection using held-out query images.

6.3 Severity Scoring

The project specification defines four weighted severity factors: defect size (30%), location (25%), defect type (25%) and detection confidence (20%).

Factor	Weight
Defect Size	30%
Defect Location	25%
Defect Type	25%
Detection Confidence	20%

Severity levels: Critical (80–100), High (60–79), Medium (40–59), and Low (0–39).

7. Dataset and Evaluation Protocol

The deployed system uses 15 MVTec categories: bottle, cable, capsule, carpet, grid, hazelnut, leather, metal_nut, pill, screw, tile, toothbrush, transistor, wood and zipper.

Evaluation Component	Protocol
Categories	15 MVTec categories
Anomaly validation/test	70% validation and 30% final test
Classification	70% support/prototype images and 30% held-out query images
Random seed	42
Threshold selection	Validation data before final test
Metrics	Accuracy, precision, recall and F1-score

8. Model Testing and Validation Results

Final hold-out evaluation was completed across all 15 categories. The results show strong performance for several categories and significant challenges for others; both are reported for transparent validation.

Metric	Anomaly Detection	Classification
Macro Accuracy	0.6158	0.5038
Macro Precision	0.9246	—
Macro Recall	0.5238	—
Macro F1-score	0.6098	0.4871

8.1 Stronger Category Results

Category	Anomaly F1	Classification F1
tile	0.9286	0.8180
bottle	0.9231	0.7265
wood	0.9231	0.6984
leather	0.9206	0.6817
hazelnut	—	0.6918

8.2 Challenging Category Results

Category	Anomaly F1	Classification F1
grid	0.0952	0.1829
screw	0.2273	0.1734
toothbrush	0.2000	—
pill	0.3571	0.2748
capsule	0.5106	0.2530
transistor	—	0.3069

8.3 Threshold-Tuning Analysis

Threshold tuning was tested for difficult categories. It improved some individual cases but did not eliminate the underlying feature-separation/classification limitations.

Category	Multiplier	Test F1 after tuning
grid	2.0	0.0667
screw	1.5	0.2740
pill	1.5	0.4842
toothbrush	1.5	0.5714
capsule	1.5	0.6279

9. System Testing

Test Area	Test Performed	Result
Authentication	Register/login and protected APIs	Passed
Role-based access	Quality Engineer and Factory Supervisor dashboards	Passed
Image upload	Supported formats and 10 MB validation	Passed
Category selection	All 15 MVTec categories available	Passed
Inspection API	Frontend/Nginx → FastAPI inspection request	Passed
Multi-category inspection	All configured categories tested after deployment fix	Passed
Database	Records stored and shown in history/analytics	Passed
Analytics	Trend, defect, severity and statistics views	Passed

Reports	PDF and CSV generation/download	Passed
Frontend	Nginx serves React app on HTTP port 80	Passed
Backend	FastAPI running in Docker	Passed
End-to-end demo	Login → inspect → store → analyze → report	Passed

10. Docker and Azure Deployment

The final deployment uses an Ubuntu-based Azure Virtual Machine. Docker and Docker Compose run the complete application stack. The MVTec dataset is stored on the VM and mounted into the backend container.

Service	Container	Purpose
db	postgres:16	Persistent PostgreSQL database
backend	visioninspect-ai-backend	FastAPI application and AI inference
frontend	visioninspect-ai-frontend	React build served by Nginx and API reverse proxy

10.1 Deployment Configuration

- Frontend is exposed on HTTP port 80 for public access.
- Nginx serves the React application and forwards /api/ requests to the backend.
- Backend connects to PostgreSQL over the Docker Compose network.
- MVTec dataset is mounted into the backend container.
- Nginx upload limit is configured for images up to 10 MB.
- Nginx API proxy timeouts accommodate CPU-based inference.
- PostgreSQL health checking and service restart configuration are enabled.

10.2 Live Application

Public deployment URL: http://20.244.18.190/

The URL is intended for the final demonstration while the Azure VM is running and the public IP remains available.

11. User Interface and Reporting

- Login and authenticated access.
- Quality Engineer dashboard with inspection and defect statistics.
- New Inspection page with upload, category and inspection mode selection.
- Inspection result display with defect, severity and quality decision.
- Inspection History with searchable records and detailed results.
- Analytics with pass rate, defect rate, critical defect rate, trends and distributions.
- Production Quality Reports with PDF and CSV export.
- Factory Supervisor dashboard for production-quality monitoring.
- Profile and settings pages.

12. Performance, Limitations and Risk Considerations

The system supports live end-to-end inspection, but inference is CPU-based on the Azure VM and is therefore slower than a GPU-backed production deployment. Model performance also varies by MVTec category.

- Inference latency is higher on CPU because feature extraction and reference comparisons are computationally intensive.
- Classification quality varies substantially between categories.
- The current detector uses pretrained feature representations and reference-image similarity rather than category-specific end-to-end fine-tuning.
- The platform is an academic/prototype inspection system and requires additional validation before safety-critical industrial deployment.
- The public demo depends on the Azure VM remaining powered on and reachable.

13. Future Enhancements

- Fine-tune domain-specific models or adopt dedicated anomaly-detection architectures.
- Evaluate PatchCore, PaDiM or teacher-student anomaly detection methods.
- Use GPU-backed inference for lower latency.
- Add batch/asynchronous inspection processing.
- Improve defect localization and segmentation explanations.
- Improve low-performing categories with category-specific training and augmentation.
- Move credentials/secrets into secure Azure configuration.
- Add HTTPS, domain name and TLS.
- Add monitoring, structured logs, automated health checks and CI/CD.

14. Final End-to-End Demonstration Procedure

12. Open the live VisionInspect AI URL.
13. Log in with a Quality Engineer account.
14. Show Dashboard statistics.
15. Open New Inspection and upload a sample MVTec image.
16. Select the matching product category.
17. Run inspection and explain anomaly, defect type, severity and quality decision.
18. Open Inspection History and show the stored record.
19. Open Analytics and explain quality trends and defect distribution.
20. Open Reports and demonstrate PDF/CSV export.
21. Optionally log in as Factory Supervisor and demonstrate role-specific navigation.

15. Milestone 4 Evidence Checklist

Evidence	Status
Live frontend on Azure	Available
FastAPI backend deployed	Available
PostgreSQL in Docker	Available
Docker Compose deployment	Available
MVTec dataset on deployment VM	Available
All 15 categories	Available
Hold-out model evaluation	Completed

Threshold-tuning analysis	Completed
Inspection history and analytics	Available
PDF/CSV reporting	Available
Two role dashboards	Available
End-to-end demonstration	Completed
Technical documentation	This document
Final presentation	To accompany submission

16. Conclusion

Milestone 4 completes the testing, deployment and documentation phase of VisionInspect AI. The platform has been containerized with Docker Compose and deployed to an Azure Linux virtual machine containing PostgreSQL, FastAPI and a React/Nginx frontend. The MVTec dataset is available to the deployed backend and the complete inspection workflow operates through the public application.

Model validation was performed using held-out data across 15 categories, with accuracy, precision, recall and F1-score reported for anomaly detection and classification. The final application also provides inspection history, manufacturing analytics, severity-based quality decisions and downloadable reports.

Overall, the implementation satisfies the Milestone 4 objectives of model testing and validation, deployment, documentation and end-to-end demonstration, while identifying clear opportunities for future improvement toward a production-grade industrial inspection solution.

17. References and Project Resources

- MVTec Anomaly Detection Dataset.
- PyTorch and Torchvision.
- FastAPI and Uvicorn.
- React.js / Vite and Nginx.
- PostgreSQL.
- Docker and Docker Compose.
- Microsoft Azure Virtual Machine.
- Git and GitHub.